

SIH26170

Burn-In Latent Defect Screening

Gap analysis and upgrade blueprint: what to fix, what to add, which models to use, and how to prove it

Prepared 1 October 2026, from a review of the project repository (preprocess, Module A, Module B, decision engine, explainability, generator, evaluation, Docker/API files) against the official problem statement.

The one-paragraph version

Your pipeline is solid engineering, but the three judged criteria (false-negative penalty, 168h MAE, explainability) are currently supported by results on **self-generated data shaped around your own thresholds**. The upgrade path below turns each claim into something defensible: a **certified recall bound**, **calibrated forecast intervals**, **exact per-part explanations**, and a **stress-test suite**, plus a neural/probabilistic model layer that adds uncertainty and physics, not just complexity.

Contents

- 1. Scorecard against the three judging criteria
- 2. Problems in the current state
- 3. Code-level bugs and quick fixes
- 4. Target architecture
- 5. Machine learning and neural network layer (Module A and Module B)
- 6. Wow-factor features: teacher-student, sequential testing, mission-life safety slope
- 7. Explainability upgrades
- 8. Data strategy: physics-based generator, public datasets, stress tests
- 9. Evaluation protocol and baselines
- 10. Roadmap and priorities
- 11. Demo storyline and judge Q&A
- Appendix A: reference code snippets; Appendix B: dependency changes

1. Scorecard against the three judging criteria

The problem statement scores three things. Everything else (MES webhooks, QR certificates, ATE parsers) is secondary and should not consume build time.

Criterion	Where you are now	Risk	Target upgrade
Anomaly detection score (false negatives penalised)	Rule layer + robust Mahalanobis + Isolation Forest; thresholds tuned on a labelled validation split; 100% recall on own synthetic test.	No statistical guarantee. About 30 test defects cannot support "99% recall". Defects were generated inside your own thresholds.	Conformal / Clopper-Pearson certified recall; harder overlapping defects; multi-seed results with confidence intervals; stress tests.
Drift prediction accuracy (MAE on hidden 168h)	Gradient boosting on log-delta using 0h/24h-derived features; MAE 0.026 on own synthetic data.	Trees cannot extrapolate for severe defects. 10/90 bands are uncalibrated. Overall MAE hides defect-only error.	Deep ensemble + GP + physics head stacked with GBM; conformalised quantile bands; report defect-only MAE and band coverage.
Explainability	Templated QA narrative; "contributions" are raw values divided by constants.	Narrative assumes every rejected part passes the static limit and saves 144h; attribution is not real attribution.	Exact Mahalanobis decomposition, counterfactual margins, nearest healthy neighbours, trajectory cone, rule trace for accepted parts too.
Robustness (implicit)	Not tested.	Contaminated lots, tiny lots, reused lot IDs, noise and missing values can all break lot-relative stats.	Dedicated stress-test suite with a slide on where it breaks and how it degrades.

2. Problems in the current state

#	Problem	Why it matters	Fix
P1	Circular synthetic data	Defect magnitudes are capped relative to the datasheet and your thresholds, and healthy parts are clipped. Near-100% recall is expected by construction; the static-limit baseline at 0% looks like a strawman.	Physics-based generator, continuous defect severity (some defects nearly healthy), held-out generator seed with shifted hyperparameters, mean $\pm$ CI over 20+ seeds.
P2	Dependence on is_defect	MCD fit, thresholds, safety slope and cost calibration all use labels. The hidden set likely has none at inference and possibly none at all.	Unsupervised-first pipeline. Define pseudo-labels on training lots from the full trajectory (robust z of 168h log-drift) and treat true labels as optional.
P3	No recall guarantee	99% is met on one validation split. Zero misses on 30 positives only bounds the true miss rate near 10% (rule of three).	Certified threshold using a one-sided Clopper-Pearson bound; generate enough calibration defects (about 300+ for 99% at 95% confidence with zero misses).
P4	96h and 168h underused	Module B sees only 0h/24h-derived features; the other columns are free supervision.	Teacher-student: teacher uses all four readings, student sees only 0h/24h.
P5	Hand-tuned score fusion	0.45/0.55 weights, sigmoid constants, +0.35 boost, reject = review + 0.25 are arbitrary and unexplainable.	Stacked meta-learner with isotonic calibration producing a real probability.
P6	Lot stats fragile	A maverick lot with 30-40% bad parts shifts the median and hides defects. Training lot IDs reuse stored stats on new data.	Per-batch robust stats, lot-level maverick detector, hierarchical Bayesian lot model.
P7	Absolute safety slope	A single $\mu\text{A/h}$ limit over-flags high-leakage lots and under-flags low ones; $k\text{-}\sigma$ has no physical meaning.	Relative slope (slope / lot median) and an Arrhenius mission-life derivation.
P8	Forecaster limits	Tree models cap predictions for rare severe defects; bands are not calibrated.	Add NN / GP / physics head; calibrate with conformalised quantile regression.
P9	Cosmetic explainability	"SHAP-compatible" values are scaled inputs; narrative text is not conditional on the actual evidence.	Exact attribution, counterfactuals, neighbours; conditional narrative.
P10	REVIEW tier ambiguity	REVIEW is counted as caught everywhere. If the official metric is binary, precision and FP counts change.	Report both mappings explicitly and justify the tier.
P11	Scope creep	ATE parsers, QR certificates, webhooks and a "millisecond hardware trip" add unvalidated claims and invite hard questions.	One appendix slide labelled as integration adapters; use "aligned with" not "compliant with" for MIL-STD/ECSS/NASA standards.
P12	No robust entry point	Hidden CSV may use different casing or lack 96h/168h columns.	Single predict.py with schema normaliser, tolerant of missing columns, writing predictions + decision + reason.

3. Code-level bugs and quick fixes

File	Issue	Fix
decision.py	composite_risk_score can exceed 1.0 (score_a + 0.35 not clipped).	Clip to [0, 1] or replace with calibrated probability.
module_b.py evaluate_drift_prediction	Chamber hours saved counts every Module B REJECT, including false positives, at 144h each.	Count hours only for true defects intercepted; report FP chamber cost separately.
preprocess.py	small_lot_threshold is never used; all lots (150-400 parts) are shrunk toward the global median/MAD.	Shrink only when n < small_lot_threshold.
preprocess.py	Seen lot_id reuses stored training stats; row loop with iterrows is slow.	Compute stats from the scoring batch; vectorise with groupby/map.
module_a.py calibrate_thresholds	Silently keeps defaults if the target recall is unreachable; reject = review + 0.25 is arbitrary.	Warn or raise; calibrate both thresholds on their own criteria.
module_a.py fit	MCD is fit on healthy rows, Isolation Forest on all rows.	Use one convention; in label-free mode fit both on all rows with robust contamination.
module_a.py rules	Rules only flag high values.	Add a low-side or two-sided rule (abnormally low leakage is also a screening signal).
evaluate.py	cm[1, 0] indexing fails if a split has one class.	confusion_matrix(..., labels=[0, 1]).
explain.py	Narrative always says the part passes the static limit and saves 144h; compares a 24h slope with a 168h-average safety slope.	Make text conditional; compare slopes on the same time basis.
README / compose / env	Hard-coded HMAC secret, "compliant" claims, ambiguous MAE units.	Env-var secrets, "aligned with", state units (μ A vs log) next to every MAE.

4. Target architecture

Keep the existing structure and add layers. Every stage should work with or without labels, and every stage should be able to say "I am not sure".

Stage	Component	Output
0	Ingest and schema normaliser (case-insensitive columns, optional 96h/168h)	Clean dataframe + data-quality flags
1	Lot-level QC: maverick-lot detector, small-lot flag	Lot health status; chooses per-batch vs prior stats
2	Feature builder: log values, drift, lot-relative z, hierarchical posterior features	Lot-normalised feature vector
3	Module A ensemble: rules, MCD, Isolation Forest, autoencoder, GMM/GP novelty, stacked and calibrated	P(defect), per-layer scores, rule trace
4	Module B: deep ensemble, GP, physics head, GBM, stacked; conformalised quantile band	168h median, calibrated 10/90 band, drift rate
5	Decision: certified thresholds; ACCEPT / REJECT / UNCERTAIN, with optional extension to 96h	Disposition + chamber hours saved
6	Explain: attribution, counterfactual, neighbours, trajectory cone, model-card caveats	Inspector report card
7	Audit: existing hash-chained trail	Traceability

5. Machine learning and neural network layer

5.1 Module A: anomaly detection

Model	What it adds	Role	Effort
Lot-conditioned autoencoder (PyTorch)	Learns the joint shape of healthy (0h, 24h, drift) features. Per-feature reconstruction error is also an explanation.	Detector layer + attribution	Medium
Deep SVDD / one-class network	Compact hypersphere around healthy embeddings; strong when defects are diverse.	Optional alternative to AE	Medium
Gaussian mixture or normalising flow on healthy features	Density-based anomaly score; handles multi-modal lot populations. Start with GMM, flow is optional.	Detector layer	Medium

Hierarchical Bayesian lot model (PyMC or NumPyro)	Lot as random effect gives principled small-lot priors and a posterior probability that a part exceeds its lot tail. Replaces manual shrinkage.	Feature builder	High
Local Outlier Factor / in-lot kNN	Cheap non-parametric local-density member for diversity.	Ensemble member	Low
Stacked meta-learner with isotonic calibration	Learns fusion weights instead of hand-tuning; outputs a calibrated probability. Needs labels or pseudo-labels.	Fusion	Low-Medium
Conformal risk control wrapper	Converts the score into a threshold with a stated recall guarantee.	Decision	Low

5.2 Module B: 168h drift forecasting

Model	What it adds	Effort
Heteroscedastic MLP (predicts mean and variance, trained with Gaussian NLL)	Per-part uncertainty that grows for unusual inputs.	Medium
Deep ensemble (5 MLPs, different seeds)	Epistemic uncertainty; typically better calibrated than a single net.	Medium
Quantile MLP (pinball loss) + conformalised quantile regression	10/90 bands with verified coverage on held-out lots.	Medium
Physics-informed head: $v(t) = v_0 \cdot (1 + \alpha \cdot (t/168)^\gamma)$	Network outputs α and γ with γ constrained positive. Extrapolates sensibly and γ is interpretable (sub-linear healthy, super-linear TDDb-like). Two readings identify only one combination of α and γ , so use an informative prior on γ learned from healthy lots.	Medium-High
Gaussian process on log-drift with lot-aware kernel	Well-calibrated uncertainty on small data; good independent check on the NNs.	Medium
Gradient boosting (existing)	Keep as a strong baseline and ensemble member.	Done
Optional pretrained tabular model (e.g. TabPFN)	Strong small-data baseline; verify installation and licence before relying on it.	Low
Stacking across the above	Weights learned on validation lots by minimising MAE; usually the final winner.	Low

Be honest about neural nets

With only two scalar inputs per part plus lot features, gradient boosting will often **match or beat** a neural net on point MAE. Do not claim otherwise without the ablation to back it.

The real value of the NN/probabilistic layer is **calibrated uncertainty, physics-consistent extrapolation for severe defects, and ensemble robustness**. Present it that way, with an ablation table showing each member's MAE and the stack's MAE.

6. Wow-factor features

6.1 Certified recall (conformal risk control)

Instead of saying "we measured 100% recall", say "on calibration defects the one-sided 95% lower bound on recall is at least 99%". Choose the highest score threshold whose Clopper-Pearson lower bound on recall reaches the target (code in Appendix A).

- Zero misses certifies 99% recall at 95% confidence only with about 300 or more calibration defects ($0.99^n \leq 0.05$). With a 1.5% defect rate that means roughly 20,000 calibration parts, so use the simulator, or report the largest recall you can certify with the defects you have.
- Certification is valid only if calibration and deployment data are exchangeable. State that assumption and show the stress test where it is violated.

6.2 Teacher-student with privileged information

Train a teacher on all four readings (0h, 24h, 96h, 168h) to label true latent defects and produce soft 168h targets. Train the student only on 0h/24h inputs to match the teacher's labels and forecasts. At inference only 0h/24h are used, so the problem-statement rule holds, but the student has learned from the whole trajectory.

6.3 Sequential testing with abstention

At 24h the system outputs ACCEPT, REJECT or UNCERTAIN (continue to 96h). At 96h it re-decides with a sequential probability ratio test on the updated posterior. This trades chamber hours against escape risk instead of only flagging at 24h, and shows the economic reasoning judges like.

6.4 Mission-life safety slope

Replace $k \cdot \sigma$ on healthy slopes with a physical definition: the drift rate at which the part would cross the datasheet limit before end of mission life. The Arrhenius acceleration factor is $AF = \exp[(E_a/k)(1/T_{\text{use}} - 1/T_{\text{stress}})]$.

Item	Value / formula
Assumed activation energy E_a	0.7 eV (illustrative; choose from JEDEC JEP122 for the dominant mechanism)
Boltzmann constant k	8.617×10^{-5} eV/K
$T_{\text{stress}} / T_{\text{use}}$	125 °C (398.15 K) / 55 °C (328.15 K)
Acceleration factor	$AF \approx 78$
Equivalent field time of 168h burn-in	$168 \text{ h} \times 78 \approx 13,000 \text{ h} \approx 1.5 \text{ years}$
Safety slope	Largest relative 168h drift such that extrapolated field drift stays below the datasheet limit at mission end; report sensitivity to E_a

6.5 Other ideas worth a slide

- **Lot-level maverick detector:** flag the whole lot when its distribution departs from historical lots, before judging individual parts.
- **Active retest:** for UNCERTAIN parts, rank which retest would reduce uncertainty most.
- **Drift monitoring across lots:** detect slow process shift (a new fab line) and trigger recalibration.
- **Digital twin simulator:** the physics generator doubles as a what-if tool in the dashboard (change E_a , lot sigma, defect rate and see the effect on recall and chamber hours).

7. Explainability upgrades

Upgrade	What the QA inspector sees	Method
Counterfactual margin	"This part would have passed if its 24h reading were at most 14.2 μA (measured 38.5 μA)."	Closed form for rule thresholds; line search for the ensemble.
Exact Mahalanobis decomposition	Bar chart of which features drove the distance.	$d^2 = \sum_i \text{diff}_i \cdot (\Sigma^{-1} \text{diff})_i$, so contributions sum exactly to the distance.
Autoencoder reconstruction error per feature	Which signal the model could not reproduce.	Per-feature squared error.
Nearest healthy neighbours	The three most similar passing parts in the same lot, plotted against the suspect.	kNN in normalised feature space.
Trajectory cone	Lot envelope, observed 0h/24h, forecast cone, datasheet limit, safety slope.	Data already prepared in explain.py.
Rule trace for accepted parts	Which checks ran and why none fired.	Log every rule with value and threshold.
Calibrated probability and uncertainty	"Defect probability 0.93 (80% band 0.81-0.98)."	Isotonic calibration + ensemble spread.
Conditional narrative and model card	Wording that depends on the real evidence; plain statement of limitations.	Template branches on actual rule firings.

8. Data strategy

8.1 Physics-based generator

Replace convenient distributions with mechanisms. Use typical literature forms and confirm exponents and activation energies against JEDEC JEP122 before quoting them.

Mechanism	Model form	Signature in the data
Healthy ageing (NBTI-like, ion drift)	Power law in time, $\Delta V \propto t^n$ with n typically about 0.15-0.3; saturating leakage drift	Small sub-linear drift
Subthreshold leakage spread	$I \propto \exp(-V_{th} / (n \cdot kT/q))$ with Gaussian V_{th} variation	Log-normal within-lot spread
TDDB-like dielectric defect	Soft breakdown, progressive then accelerating leakage (E-model / power-law time dependence)	Looks almost normal at 24h, accelerates later
Electromigration-like	Black's equation: $MTTF = A \cdot J^{(-n)} \cdot \exp(E_a/kT)$	Steady super-healthy drift rate
Mobile-ion contamination	Bias-temperature dependent threshold shift	Early step then saturation
ESD / oxide damage	High initial offset, moderate drift	In-spec but far above lot median

- Draw defect **severity from a continuous distribution** so some defects are nearly healthy. A few unavoidable false negatives make the recall curve honest and the conformal story meaningful.
- Add tester repeatability noise, temperature offset at readout, missing readings, and lots with heavy tails.
- Use a **held-out generator configuration** (different lot spread, defect mix and noise) for the final test, to measure domain shift.
- Optional: an ngspice/PySpice Monte Carlo of subthreshold leakage with V_{th} variation to justify the log-normal spread.

8.2 Public data and references

Real burn-in leakage data is largely proprietary, so say so up front. These sources are useful for validating methods; I have not re-verified the live links or licences, so check them before citing them.

Source	What it offers	Use in your project	Caveat
NASA Prognostics Data Repository (Ames)	Accelerated ageing sets: MOSFET thermal overstress, IGBT, capacitor electrical stress	Validate Module B drift forecasting on real degradation trajectories	Different devices and signals than leakage
UCI SECOM	About 1,567 semiconductor process samples, pass/fail label, heavy imbalance	Test Module A on real, messy, imbalanced data	Process data, not burn-in time series
NASA C-MAPSS / N-CMAPSS	Run-to-failure degradation trajectories	Early-prediction methodology and uncertainty calibration	Engine domain
CALCE and PHM Society challenge data	Accelerated-ageing and prognostics datasets	Additional drift-forecast validation	Check licences
JEDEC JEP122, JESD22-A108; AEC-Q001; MIL-STD-883 Method 1015; MIL-PRF-38535	Failure-mechanism models, operating-life test, PAT, burn-in method	Justify generator physics, Arrhenius factor and PAT rules	Some documents are paywalled
Organisers' sample data (if provided)	The actual schema and distribution	Final schema normaliser and sanity checks	Often small

8.3 Stress-test suite

Scenario	What changes	Expected behaviour
Maverick lot (35% bad)	Lot median shifts toward defects	Lot-level detector flags the lot and falls back to historical priors
Tiny lot (15-20 parts)	Few samples for lot stats	Hierarchical prior; wider uncertainty routes to REVIEW, never silent ACCEPT
Lot baseline shift	New fab line, higher base leakage	Per-batch robust stats unaffected
Noise $\times 3$	Tester repeatability worse	False positives rise gracefully; show the curve
Missing 0h or 24h	NaN values	Imputation + abstain, flagged in output
Parts at the threshold	Boundary cases	UNCERTAIN / REVIEW tier
Reused or unseen lot IDs	ID collisions with training	Batch stats used; no stale lot reuse

Label-free mode	No is_defect column	Pseudo-labels from trajectory; same pipeline runs
-----------------	---------------------	---

9. Evaluation protocol and baselines

9.1 Baselines judges will recognise

Module	Baselines to include
A: detection	Static datasheet limit; global z-score; classic static PAT (fixed sigma limits); per-lot DPAT; Isolation Forest; LOF; One-Class SVM; autoencoder; supervised XGBoost as an upper bound; your full ensemble
B: forecasting	Carry-forward (v168 = v24); linear extrapolation; power-law physics fit; Ridge; Huber; gradient boosting; GP; deep ensemble; stacked model

9.2 What to report

Area	Metrics
Detection	Recall with Clopper-Pearson interval, false-negative count, precision, cost (50-FN + FP), PR-AUC, recall at fixed false-positive rate; both REVIEW mappings
Forecasting	MAE in μA overall and defect-only, log-space MAE, R^2 , coverage of the 10-90 band (target about 80%), band width
Economics	Chamber hours saved on true defects only, net of false-positive cost
Protocol	Lot-grouped splits, 20+ seeds, held-out generator configuration, ablation of every layer, runtime per part

10. Roadmap and priorities

Phase	Work	Deliverable	Score impact	Effort
0	Fix listed bugs; schema normaliser; predict.py	Pipeline that cannot crash on the hidden file	Protects all metrics	Low
1	Physics generator, severity continuum, held-out seed, stress suite	Credible data and a robustness slide	FN claim credibility	Medium
2	Conformal certified threshold; stacked calibrated fusion	Recall guarantee	FN penalty	Low-Medium
3	Teacher-student; deep ensemble; GP; quantile MLP + CQR; stack	Lower defect-only MAE, calibrated bands	MAE	Medium
4	Autoencoder, GMM, hierarchical lot model	Diverse detectors; label-free mode	FN penalty, robustness	Medium-High
5	Counterfactuals, exact attribution, neighbours, trajectory cone	Inspector report card v2	Explainability	Medium
6	Mission-life slope; sequential 24h/96h decision	Physical safety slope; chamber-hours story	Judge confidence	Low-Medium
7	Multi-seed benchmark, ablation tables, trim integration material, pitch	Final report and demo	All	Medium

If time is short, do phases 0, 1, 2 and 5 first: robustness, a certified recall bound, and strong explanations move the judged criteria more than additional model types.

11. Demo storyline and judge Q&A

Beat	On screen	Message
1	A part at 45 μA in a 10 μA lot, below the 50 μA limit	Static limits pass it; the lot-aware system does not
2	Trajectory cone and counterfactual	"Would pass if $24\text{h} \leq X$ "; the forecast crosses the limit before mission end
3	Recall bound slide	Certified $\geq 99\%$ recall at 95% confidence on calibration data
4	Stress-test grid	A 35% contaminated lot and tiny lots are handled or

		escalated
5	Ablation and baselines table	Each layer earns its place; honest NN vs GBM comparison
6	Chamber hours and mission-life slope	Economics and physics, not just ML

Likely question	Strong answer
Your data is synthetic. Why trust it?	Generator follows published failure-mechanism forms, uses continuous severity, and the final test uses a different configuration. We also validated forecasting on public degradation data and show where the method breaks.
How do you guarantee 99% recall?	We certify a lower confidence bound on recall via Clopper-Pearson on calibration defects, and state the exchangeability assumption and the stress test where it fails.
Why not just a neural network?	On two inputs, boosting is competitive for point error. We use networks for calibrated uncertainty and physics-consistent extrapolation, and the stack's ablation shows each member.
What if a lot is mostly bad?	A lot-level detector flags it and the system falls back to historical priors instead of trusting the contaminated median.
Where does the safety slope come from?	From mission life through the Arrhenius acceleration factor, with sensitivity to activation energy, not an arbitrary $k \cdot \sigma$.
What without labels?	Pseudo-labels from the full training trajectories; the same pipeline runs and we report how results change.
Why a REVIEW tier?	False negatives cost far more than retests; we report results under both the binary and three-tier mappings.

Appendix A: reference code snippets

Starting points, not tested against your repository. Adapt names and shapes to your feature arrays.

A.1 Certified recall threshold

```
import numpy as np
from scipy.stats import beta

def recall_lower_bound(k, n, conf=0.95):
    # one-sided Clopper-Pearson lower bound on recall
    return 0.0 if k == 0 else beta.ppf(1 - conf, k, n - k + 1)

def pick_threshold(scores_defect, target=0.99, conf=0.95):
    s = np.sort(np.asarray(scores_defect))[:-1] # defect scores, high to low
    n = len(s)
    for t in s:
        # highest feasible threshold
        k = int((s >= t).sum())
        if recall_lower_bound(k, n, conf) >= target:
            return t
    return None # too few calibration defects to certify this target
```

A.2 Heteroscedastic MLP for 168h log-delta

```
import torch, torch.nn as nn

class HeteroMLP(nn.Module):
    def __init__(self, d_in, h=64):
        super().__init__()
        self.body = nn.Sequential(nn.Linear(d_in, h), nn.SiLU(),
                                   nn.Linear(h, h), nn.SiLU())
        self.mu = nn.Linear(h, 1)
        self.logvar = nn.Linear(h, 1)
    def forward(self, x):
        z = self.body(x)
        return self.mu(z).squeeze(-1), self.logvar(z).squeeze(-1)

def nll(mu, logvar, y):
    return (0.5 * torch.exp(-logvar) * (y - mu) ** 2 + 0.5 * logvar).mean()

# deep ensemble: train 5 of these with different seeds, average mu,
# combine variance = mean(exp(logvar)) + var(mu across members)
```

A.3 Autoencoder anomaly score with per-feature explanation

```
class AE(nn.Module):
    def __init__(self, d_in, z=3):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(d_in, 16), nn.SiLU(), nn.Linear(16, z))
        self.dec = nn.Sequential(nn.Linear(z, 16), nn.SiLU(), nn.Linear(16, d_in))
    def forward(self, x):
        return self.dec(self.enc(x))

# train on lot-normalised healthy (or all, in label-free mode) feature rows
err = (model(x) - x) ** 2 # shape [n, d_in]
score = err.sum(dim=1) # anomaly score
# err[i] shows which feature the model failed to reproduce -> explanation
```

A.4 Conformalised quantile regression (10/90 band)

```
def cqr_margin(y_cal, q_lo_cal, q_hi_cal, alpha=0.2):
    s = np.maximum(q_lo_cal - y_cal, y_cal - q_hi_cal) # conformity scores
    n = len(s)
    k = int(np.ceil((n + 1) * (1 - alpha)))
    return np.sort(s)[min(k, n) - 1]

# calibrated interval for new parts: [q_lo - m, q_hi + m]
# calibrate on validation LOTS, not random rows, to respect lot grouping
```

A.5 Exact Mahalanobis decomposition

```
diff = x - mu_healthy          # [d]
contrib = diff * (cov_inv @ diff)  # sums to squared distance
# rank features by contrib; show the top 2-3 to the inspector
```

Appendix B: dependency changes

Package	Purpose	Note
torch (CPU build)	Autoencoder, heteroscedastic MLP, deep ensemble	Heavy for Docker; use a CPU wheel and pin the version
shap	Attribution for tree models and stacked scores	Optional if exact decompositions are enough
pymc or numpyro	Hierarchical lot model	Optional; start with the simpler robust per-batch stats
scikit-learn (existing)	GP regression, GMM, LOF, One-Class SVM, isotonic calibration	Already installed
Drop lightgbm or xgboost	Keep only the one you actually use	Smaller image, faster builds